



RESEARCH INC

Technical Architecture

Trend-Following Vaults and a Portfolio Allocation Tool on Stellar

STELLAR BUILD AWARD SUBMISSION · JUNE 2026 · WWW.ETESIA.COM

1 Introduction

Etesia brings systematic, institutional-grade asset management to Stellar by integrating the network's core DeFi and wallet building blocks. We are building two things: new products that supply genuinely diversifying return streams, and a free tool that lets users allocate across them. Etesia already runs a live systematic vault on Hyperliquid; this project ports and extends that infrastructure to Stellar.

Real diversification needs return streams that are not correlated, and on Stellar today the investable universe is almost entirely directional crypto beta: most assets move with Bitcoin. The new products supply what is missing. They are systematic trend-following vaults, which carry a structurally low and often negative correlation to buy-and-hold crypto (the live strategy runs near -0.22 correlation to Bitcoin). The vaults are the diversifying building blocks; the Portfolio Builder is the tool that allocates across them. This document describes the technical architecture and the Stellar protocol integrations covered by this submission:

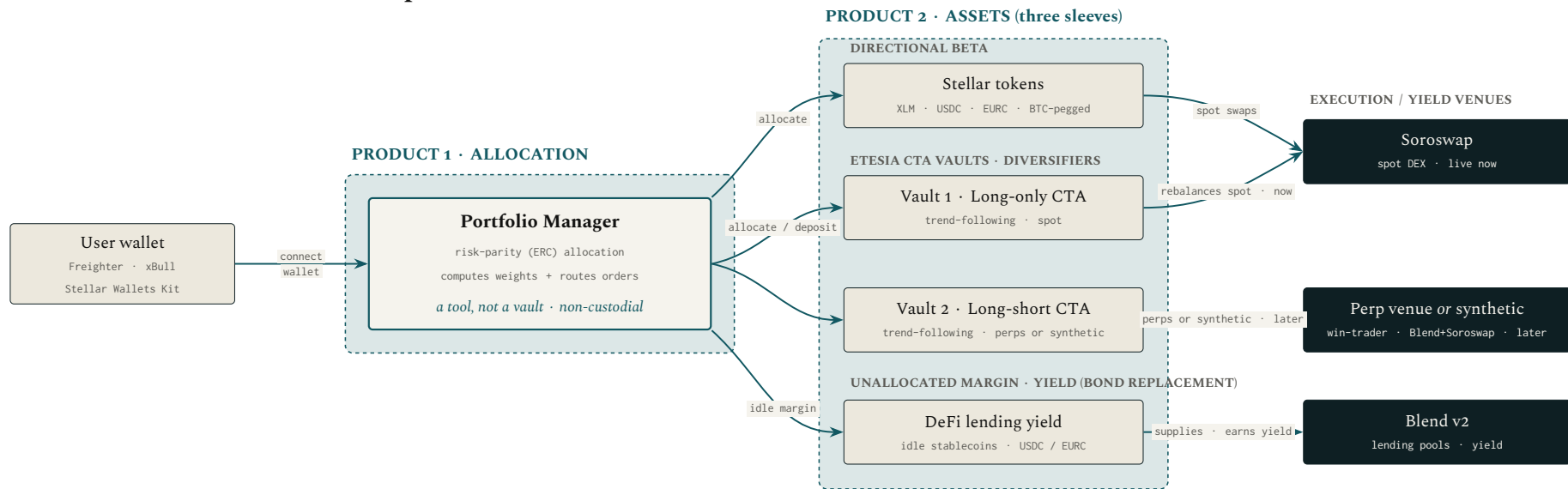
1. **Long-only Trend-Following Vault, on Soroswap:** a spot trend-following vault over Stellar assets, long while an asset trends up and rotated into stablecoins otherwise. All execution runs through Soroswap; stablecoins held during flat phases earn yield in Blend v2 lending pools. It needs only spot DEX liquidity, no perpetuals, so it ships first.
2. **Long-Short Systematic Vault:** the full long and short CTA, ported from the strategy Etesia runs live on Hyperliquid and built behind a *venue-agnostic execution interface*. One trend-following strategy drives two interchangeable execution paths: native perpetuals via whichever Stellar perp venue reaches mainnet first (win-trader, formerly `stellars.finance`, is the primary candidate; Zenex and Noether are monitored as alternatives), or synthetic short exposure constructed atomically via Blend v2 and Soroswap. Signal generation and execution run from our off-chain Python infrastructure; the venue is a thin adapter, not a dependency, so the vault does not hinge on any single third-party venue going live.

3. **Portfolio Builder, a tool and not a vault:** a non-custodial allocation tool that lets users construct a diversified, risk-balanced portfolio across Etesia vaults and other Stellar assets, using the risk parity (Equal Risk Contribution) allocation engine. Custody stays in the user's wallet and in the underlying vaults; the tool only computes and routes allocations.

All three share the same architectural principle: **off-chain intelligence, on-chain (or venue-side) custody and settlement**. Quantitative computation (covariance estimation, allocation, signal generation, execution scheduling) is too heavy for on-chain execution and runs on our Python infrastructure, the same stack that operates our live Hyperliquid vault. Funds never touch our servers: custody sits in Soroban contracts, in the underlying vaults, or in the venue's vault primitive, and the off-chain engine can only propose actions that pass hard-coded on-chain constraints. A compromised backend cannot move funds; the worst case is a missed rebalance.

The map on the following page summarizes this picture end to end: the two products, the three allocation sleeves of the asset universe, and the venue on which each sleeve executes.

Etesia on Stellar · Product Map



Custody never leaves the user. The Portfolio Manager is a non-custodial tool: it only computes risk-parity (Equal Risk Contribution) weights and routes orders. Funds stay in the user's wallet and in the vaults' Soroban / venue contracts. Vault 1 trades spot on Soroswap; Vault 2 runs one strategy over two interchangeable paths, native perpetuals (win-trader, formerly stellars.finance) or synthetic shorts built from Blend v2 + Soroswap, so it does not depend on any single venue going live. The third sleeve puts unallocated margin to work in Blend v2 lending pools, a DeFi yield substitute for the traditional bond / cash allocation rather than leaving it idle.

2 Definitions

Term	Definition
Portfolio Builder	Non-custodial allocation tool that constructs a diversified, risk-balanced portfolio across Etesia vaults and other Stellar assets. A tool, not a vault: custody stays in the user's wallet and in the underlying vaults.
Risk Parity / ERC	Equal Risk Contribution allocation: portfolio weights chosen so each asset contributes the same share of total portfolio variance. This is the allocation method inside the Portfolio Builder.
Trend Following	Systematic trend-following signal built from multiple proprietary oscillators, driving the vaults: long/flat targets for the long-only vault, long/short targets for the long-short vault.
Vault	Contract holding strategy funds. The long-only vault is a Soroban smart contract; the long-short vault uses either the perp venue's vault primitive (native-perp path) or an Etesia Soroban vault (synthetic-short path). The Soroban vaults implement SEP-56 tokenized vault semantics.
Allocation Engine	Python service computing covariance estimates, ERC weights and concentration caps that power the Portfolio Builder.
Signal Engine	Python service computing the trend-following signal and target positions for the vaults.
Orchestrator	Backend service bridging the engines and the Stellar network; builds, signs and submits rebalance plans.
Executor	The authorized key allowed to call <code>execute_rebalance</code> on the vault. Holds no withdrawal rights.
TWAP	Time-weighted average price computed from allowlisted Soroswap pools.
SEP-41 / SEP-56	Stellar token interface standard / tokenized vault standard on Soroban.
Native perp venue	A perpetual futures exchange on Stellar, used for the long-short vault's native-perp path (win-trader, formerly <code>stellars.finance</code> , primary; Zenex and Noether monitored as alternatives).
Synthetic short	Directional short exposure constructed atomically via Blend v2 (collateral deposit + borrow) and Soroswap (swap); the long-short vault's second, venue-independent execution path.
CTA	Commodity Trading Advisor style systematic strategy; here, systematic trend following, spot for the long-only vault and on perpetual futures or synthetic equivalents for the long-short vault.

3 Integration Summary

This submission integrates four protocols from the SCF Integration List, listed first below, alongside complementary ecosystem components. Each headline integration is named explicitly with its team and its role in the architecture.

Building Block	Role	On List	Tranche
Soroswap (PaltaLabs)	TWAP pricing read from pools as on-chain execution reference, plus swap execution for vault rebalancing; also the swap leg of the long-short vault's synthetic-short path. Aggregates Soroban and classic liquidity.	Yes	T1, T2, T3
Blend v2 (Script3)	Yield destination for stablecoins held during the long-only vault's flat phases; also the collateral-deposit and borrow legs of the long-short vault's synthetic-short path. Cross-contract deposit, borrow and withdraw; yield accrues to vault NAV.	Yes	T2, T3
Stellar Wallets Kit (Creit Tech)	Wallet connectivity (Freighter, xBull); trustline auto-setup for SEP-41 vault shares.	Yes	T2
DeFindex (PaltaLabs)	Composability: vaults expose the DeFindex Strategy interface (invest, unwind, harvest) so the Portfolio Builder and any DeFindex-integrated app can consume them.	Yes	T3
SEP-56 / SEP-41 (OpenZeppelin Stellar Suite)	Tokenized vault standard and share tokens; NAV accounting.	No	T1
Reflector Oracle	Price deviation and staleness circuit breaker against Soroswap TWAP.	No	T1
Phoenix Protocol	Secondary swap path, weighted AMM.	No	T2
Native perp venue	Perpetual futures venue for the long-short vault's native-perp path (win-trader primary; Zenex, Noether alternatives). Not required for the synthetic-short path.	No	T3

3.1 Integration detail: Soroswap

Soroswap's routing API, which aggregates all Stellar liquidity (Soroban and classic), is integrated at two levels.

- **TWAP pricing.** Time-weighted average prices computed from allowlisted Soroswap pools serve as the on-chain reference for NAV computation and execution bounds. The vault contract reads TWAP directly; the executor never supplies prices. TWAP sets `min_amount_out` on every rebalance leg.
- **Swap execution.** Vault rebalancing routes its swap legs through Soroswap's aggregated liquidity. Each leg specifies `{adapter, asset_in, asset_out, amount_in, min_amount_out}`; the Soroswap adapter translates this into calls against the Soroswap router contract. Slippage bounds are TWAP-derived and enforced on-chain, so a manipulated pool causes the swap to fail rather than fill badly.

```
// adapter::swap turns a verified leg into a Soroswap router call. By this point
// execute_rebalance has already floored min_amount_out against the on-chain TWAP,
// so the router call carries a hard bound the proposer could not loosen.
fn swap(e: &Env, leg: &Leg) -> i128 {
    soroswap::Router::new(e, &leg.adapter).swap_exact_tokens_for_tokens(
        &leg.amount_in,
        &leg.min_amount_out, // hard on-chain slippage bound
        &vec![e, leg.asset_in.clone(), leg.asset_out.clone()],
    )
}
```

```

        &e.current_contract_address(), // funds stay in the vault
        &(e.ledger().timestamp() + SWAP_TTL),
    )
}

```

Every rebalance cycle (weekly plus signal and drift triggers) generates recurring, organic swap volume through Soroswap.

3.2 Integration detail: Blend v2

Blend v2 lending pools are the yield destination for stablecoins held while the long-only vault is in a flat phase. Instead of parking idle USDC or EURC, the orchestrator routes it into Blend.

- **Deposit flow.** When the strategy rotates an asset to stablecoins, the orchestrator deposits the surplus into the Blend pool via cross-contract call.
- **Withdraw flow.** When the strategy re-enters and the risky sleeve scales up, the orchestrator withdraws from Blend, then executes the rebalance plan.
- **Yield accrual.** Blend lending yield accrues to the vault's total managed assets, increasing NAV and share price for all depositors.

```

// Idle stablecoins -> Blend v2 supply. The resulting position is read back by
// nav() via blend::deposited_value(), so accrued yield lifts share price directly.
fn park_in_blend(e: &Env, amount: i128) {
    let pool = blend_pool(e);
    let req = Request { request_type: SUPPLY, address: usdc(e), amount };
    blend::Pool::new(e, &pool)
        .submit(&e.current_contract_address(), // from (the vault)
                &e.current_contract_address(), // spender
                &e.current_contract_address(), // to (vault holds the bTokens)
                &vec![e, req]);
}
// withdraw mirrors this with request_type = WITHDRAW before a re-entry rebalance.

```

This follows the same submit-with-Request pattern used by established Blend consumers (Beans, DeFindex, Freightier, Meru, YieldBlox).

3.3 Integration detail: DeFindex

Each Etesia vault exposes the DeFindex Strategy interface, making it pluggable into any DeFindex vault deployed through the DeFindex Factory:

- `invest(amount)` deposits into the strategy;
- `unwind(amount)` withdraws proportionally;
- `harvest()` compounds Blend yield and rebalancing gains.

```

// The vault implements the DeFindex Strategy trait, so any DeFindex vault (or the
// Portfolio Builder) can allocate into it with no Etesia-specific integration.
pub trait Strategy {
    fn invest(e: Env, from: Address, amount: i128) -> i128; // -> shares minted
    fn unwind(e: Env, to: Address, shares: i128) -> i128;    // -> assets, pro-rata
    fn harvest(e: Env) -> i128; // compound Blend yield + rebalance gains into NAV
}
// invest/unwind reuse the same deposit/redeem accounting shown above, so DeFindex
// callers inherit the identical share-price and rounding guarantees.

```

Any DeFindex-integrated wallet or application, including the Portfolio Builder itself, can then offer Etesia strategies to users with no extra integration work.

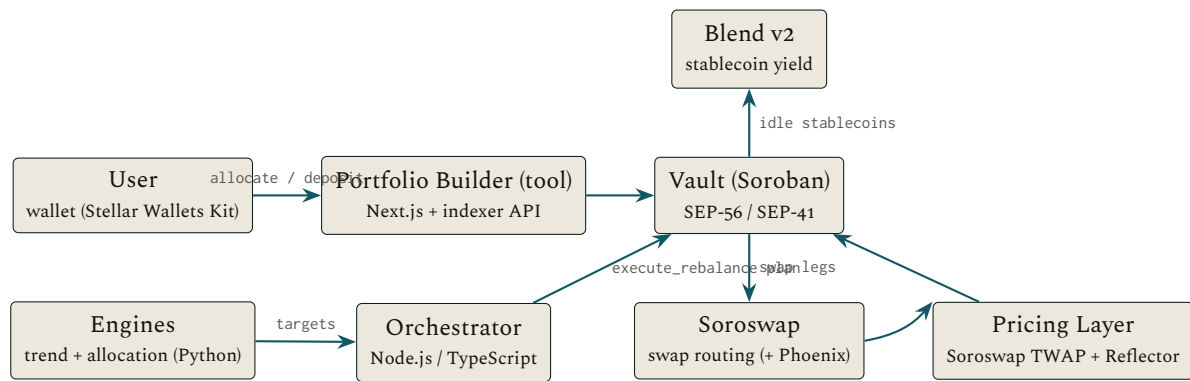
3.4 Integration detail: Stellar Wallets Kit

Frontend wallet connectivity uses Stellar Wallets Kit (@creit-tech/stellar-wallets-kit), supporting Freightier, xBull and any other Stellar-compatible wallet. It handles authentication, transaction signing, and automatic trustline setup for SEP-41 vault share tokens on first deposit, so a user connects and deposits in a single flow.

4 Architecture Constraints

- **Non-custodial by construction.** Users deposit into contracts, not accounts we control. Withdrawals are permissionless and never depend on our backend being online.
- **Operator as proposer, not owner.** The off-chain engine proposes rebalances. The vault independently verifies every proposal against on-chain invariants (asset allowlists, slippage bounds, notional caps, cooldowns, pause state) and rejects anything outside them. A compromised backend cannot drain funds; the worst case is a missed or suboptimal rebalance.
- **Soroban compute limits.** Covariance estimation and ERC optimization involve matrix operations unsuited to on-chain execution. They run off-chain; only verification logic, accounting and transfers run on-chain.
- **Transparency.** Target weights are published on-chain before execution, and the allocation methodology is documented, so any third party can verify that executed trades match the published policy.
- **Venue-agnostic execution (long-short vault).** The long-short strategy is independent of its execution path: the signal engine, risk controls and position sizing are identical whether it trades a native perp venue (win-trader primary; Zenex, Noether alternatives) or synthetic shorts built from Blend v2 and Soroswap. The venue is a thin adapter behind an internal interface, not a dependency, so if no perp venue reaches mainnet the synthetic path delivers the same exposure without touching strategy code.

5 System Overview: Long-Only Vault and Portfolio Builder



The flow is: (1) the trend-following signal engine produces long/flat targets for the vault, and the allocation engine produces risk parity weights for the Portfolio Builder; (2) the Orchestrator diffs targets against current vault holdings, publishes the new targets on-chain, and submits a structured rebalance plan; (3) the vault verifies the plan against its invariants and executes swap legs through Soroswap, depositing idle stablecoins into Blend v2 during flat phases; (4) the indexer captures emitted events and serves performance data to the Portfolio Builder.

6 Allocation Engine (Risk Parity)

The allocation engine is the risk parity (Equal Risk Contribution) engine that powers the Portfolio Builder tool. Given a set of Etesia vaults and other Stellar assets, it computes weights so that each component contributes equally to total portfolio risk. The tool routes these allocations to the user's wallet and the underlying vaults; it never takes custody.

6.1 Covariance estimation

Let $r_t \in \mathbb{R}^n$ be the vector of daily log returns of the allocation universe. Covariance is estimated with a standard exponentially weighted estimator, which weights recent observations more heavily:

$$\Sigma_t = \lambda \Sigma_{t-1} + (1 - \lambda) r_t r_t^\top, \quad \lambda \in (0, 1).$$

On top of this base estimate we apply our own multi-horizon blending and shrinkage to obtain the covariance estimate $\hat{\Sigma}$ used downstream, calibrated by walk-forward validation. That refinement is proprietary and not detailed here.

6.2 Equal Risk Contribution solver

The risk contribution of asset i at weights w is

$$RC_i(w) = \frac{w_i (\hat{\Sigma} w)_i}{\sqrt{w^\top \hat{\Sigma} w}}, \quad \sum_{i=1}^n RC_i(w) = \sigma_p(w).$$

ERC weights solve $RC_i(w) = RC_j(w) \forall i, j$ subject to $w_i \geq 0$, $\sum_i w_i = 1$. The engine solves this with a fast, deterministic iterative method robust for the universe sizes involved (10 to 30 assets), with a documented convergence tolerance on the dispersion of risk contributions. A hard concentration cap (e.g., 25% per asset) is applied as a post-constraint with renormalization.

6.3 Volatility targeting overlay

Each allocation can carry a fixed annualized volatility target σ^* (tiers, e.g., 10% / 20% / 30%). With $\hat{\sigma}_p = \sqrt{w^\top \hat{\Sigma} w}$ the estimated portfolio volatility, the risky sleeve is scaled by

$$s = \min\left(1, \frac{\sigma^*}{\hat{\sigma}_p}\right),$$

allocating fraction s to the ERC basket and $1 - s$ to stablecoins. Idle stablecoins are deposited into Blend v2 lending pools rather than held flat, so the defensive sleeve still earns yield (see the Blend v2 integration detail). This keeps realized volatility near the target in stressed regimes while remaining fully invested in calm ones.

6.4 Rebalancing policy

Rebalancing is hybrid: a weekly calendar rebalance plus a drift trigger that fires when any asset weight deviates from target by more than a threshold (e.g., 10% relative). Both paths produce the same artifact: a target weight vector published on-chain, followed by a rebalance plan whose legs are constructed to minimize turnover (netting buys against sells) and respect per-leg slippage bounds.

7 Trend-Following Engine

The trend-following engine generates the target positions that drive the vaults, porting the systematic model Etesia runs live on Hyperliquid. The signal aggregates multiple proprietary oscillators into a single directional target per asset; their construction is proprietary and not detailed here. What matters for this architecture is that output, a directional target per asset, and how it maps to each vault mandate:

- **Long-only vault (Vault 1).** The engine targets *long* an asset while it trends up and *flat* otherwise, with the flat allocation rotated into stablecoins that earn yield in Blend v2. This vault trades spot on Soroswap and needs no perpetuals, so it ships first.
- **Long-short vault (Vault 2).** The engine targets *long* in an uptrend and *short* in a downtrend, the full CTA, executed through a venue-agnostic interface: native perpetuals (win-trader primary; Zenex, Noether alternatives) or synthetic shorts built from Blend v2 and Soroswap. The strategy output is identical across paths.

Position sizes are scaled to a portfolio volatility target and bounded by the liquidity limits described later. Signal parameters are calibrated by walk-forward validation, never in-sample. The live strategy's realized correlation to Bitcoin is near -0.22 , which is what makes it a genuine diversifier for an otherwise directional Stellar universe.

8 On-Chain Design (Soroban)

On Soroban the vault contract is the trust anchor, and the off-chain engine is only a *proposer*. Every protection that matters lives in the Rust contract, so it can refuse any unsafe action on its own even if the backend is fully compromised. Soroban is explicit about four things the contract has to get right, and we document each below: **storage** (typed, tiered, with explicit TTLs), **authorization** (per-entry-point `require_auth`), **events** (the on-chain audit trail), and **invariants** (the checks that gate every state change). The guiding rule is that the contract is self-sufficient: a valid signature from the executor key is necessary but never sufficient, since a proposal also has to pass every invariant below.

8.1 Contract storage layout

Soroban storage is typed and split into tiers with explicit time-to-live (TTL): *instance* storage for small contract-wide config (read cheaply on every call, extended with the contract), and *persistent* storage for per-account balances and allowlist membership. Keys are a single contracttype enum, so every access is type-checked at compile time.

```
#[contracttype]
pub enum DataKey {
    Admin, Executor, Guardian, // role addresses (instance)
    Paused, // bool kill-switch (instance)
    RiskParams, // slippage bps, caps, cooldown (instance)
    Nonce, // monotonic rebalance counter (instance)
    LastRebalance, // ledger timestamp of last exec (instance)
    TotalShares, // S, total SEP-41 share supply (instance)
    Assets, // Vec<Address>, capped NAV universe (instance)
    TargetEpoch, // current published-weights epoch (instance)
    Balance(Address), // s_i, shares held by an account (persistent)
    AssetOk(Address), // asset allowlist membership (persistent)
    AdapterOk(Address), // DEX-adapter allowlist membership (persistent)
}

// reads are tier-explicit; touched entries are TTL-bumped to avoid expiry
fn paused(e: &Env) -> bool {
    e.storage().instance().get(&DataKey::Paused).unwrap_or(false)
}

fn shares_of(e: &Env, a: &Address) -> i128 {
    e.storage().persistent().get(&DataKey::Balance(a.clone())).unwrap_or(0)
}
```

Allowlist membership is a keyed O(1) read on the hot path (`AssetOk`, `AdapterOk`); a separate admin-maintained, length-capped `Assets` vector (the 10–30 asset universe) backs NAV enumeration, so the contract only ever iterates a bounded, admin-set list, never unbounded user input.

8.2 Share accounting and NAV (SEP-56)

The vault implements SEP-56 tokenized-vault semantics with SEP-41 share tokens: `deposit`, `mint`, `withdraw`, `redeem`, the matching `preview_*/convert_*` views, and the standard events. Shares are issued in proportion to net asset value. With A the total managed assets (valued through the pricing layer), S the share supply, and b_k the vault’s balance of asset k :

$$\text{share price} = \frac{A}{S}, \quad \text{mint: } \Delta S = \left\lfloor \frac{aS}{A} \right\rfloor, \quad \text{redeem leg } k: \Delta b_k = \left\lfloor \frac{sb_k}{S} \right\rfloor,$$

with 1:1 bootstrap issuance when $S = 0$. All rounding favors the vault (issuance rounds down; the share burn for an exact-asset withdraw rounds up) so accounting dust can never be extracted by a depositor. NAV is computed by the contract itself, never supplied by the caller: it sums each allowlisted holding at the on-chain TWAP, plus stablecoins on deposit in Blend v2, whose accrued yield therefore lifts the share price for every holder.

```
// A = NAV. Priced on-chain via the pricing layer; the caller supplies nothing.
fn nav(e: &Env) -> i128 {
    let vault = e.current_contract_address();
    let mut total: i128 = 0;
    for asset in assets(e).iter() { // bounded admin-set universe
        let bal = token::Client::new(e, &asset).balance(&vault);
        total += bal * pricing::twap(e, &asset) / PRICE_SCALE;
    }
    total + blend::deposited_value(e) // Blend supply accrues to NAV
}
```

```

pub fn deposit(e: Env, from: Address, assets_in: i128) -> i128 {
    from.require_auth(); // depositor authorizes the pull
    if paused(&e) { panic_with_error!(&e, Error::Paused); }
    if assets_in <= 0 { panic_with_error!(&e, Error::ZeroAmount); }

    let s = total_shares(&e); // S
    let a = nav(&e); // A, priced before the inflow
    let shares = if s == 0 { assets_in } // 1:1 bootstrap
                  else { assets_in * s / a }; // floor(a*S/A): rounds to vault
    if shares <= 0 { panic_with_error!(&e, Error::ZeroShares); }

    token::Client::new(&e, &underlying(&e))
        .transfer(&from, &e.current_contract_address(), &assets_in);
    mint_shares(&e, &from, shares); // S += shares; Balance(from) +=
    e.events().publish((symbol_short!("deposit"), from), (assets_in, shares, a, s));
    shares
}

// Pro-rata IN-KIND redemption: a proportional slice of EVERY vault asset.
// No oracle is consulted, so withdrawal never depends on price or backend liveness.
pub fn redeem(e: Env, to: Address, shares: i128) {
    to.require_auth(); // burn only the caller's shares
    let s = total_shares(&e);
    if shares <= 0 || shares > shares_of(&e, &to) {
        panic_with_error!(&e, Error::BadShares);
    }
    let vault = e.current_contract_address();
    for asset in assets(&e).iter() {
        let bal = token::Client::new(&e, &asset).balance(&vault);
        let out = bal * shares / s; // floor(s*b_k/S): rounds to vault
        if out > 0 { token::Client::new(&e, &asset).transfer(&vault, &to, &out); }
    }
    burn_shares(&e, &to, shares); // S -= shares; Balance(to) -=
    e.events().publish((symbol_short!("withdraw"), to), shares);
}

```

The exact-asset SEP-56 withdraw (burn $\lceil aS/A \rceil$ shares for a requested asset amount) is also implemented, but because it has to size the burn from NAV it follows the deposit availability rule: usable whenever NAV can be priced. The pro-rata redeem path needs no price at all and is therefore the always-available exit.

8.3 Authorization model

Every entry point gates on `require_auth` for exactly one role. The executor key that proposes trades holds no transfer authority, so theft of that key cannot move user funds, it can only attempt rebalances, which still have to clear every invariant.

Role	Authority (enforced via <code>require_auth</code>)
User	Permissionless deposit, withdraw, redeem of own shares.
Executor	May call <code>execute_rebalance</code> only; no transfer authority.
Guardian	Pause / unpause; emergency de-risk into stablecoins.
Admin	Updates allowlists and risk parameters under timelock.

```

fn require_role(e: &Env, key: DataKey) { // generic role gate
    let who: Address = e.storage().instance().get(&key).unwrap();
    who.require_auth();
}
// deposit / withdraw / redeem -> caller.require_auth() (any user, own funds)
// execute_rebalance -> require_role(e, DataKey::Executor)
// pause / emergency_derisk -> require_role(e, DataKey::Guardian)
// set_allowlist / set_params (timelocked) -> require_role(e, DataKey::Admin)

```

8.4 Events

Beyond the SEP-56 Deposit/Withdraw events, the vault publishes its target weights *before* execution and a per-rebalance record *after*, so the full policy trail is reconstructable from the ledger alone.

```

// published before any trade: commits the engine to a target the ledger records
e.events().publish((symbol_short!("targets"), epoch), weights_hash);
// emitted after a successful rebalance: ties execution to the committed plan
e.events().publish((symbol_short!("rebal"), executor),
    (nonce, plan_hash, legs_len));

```

An off-chain indexer (PostgreSQL) consumes these to serve performance data to the Portfolio Builder. Because targets are published before execution, any third party can verify that the executed legs match the committed policy.

8.5 execute_rebalance: plan format and on-chain verification

The executor submits a structured plan {nonce, deadline, target_weights_ref, legs[]}, where each leg is {adapter, asset_in, asset_out, amount_in, min_amount_out}. The contract re-derives everything it can and rejects the whole plan on any failed check, so an incoherent or malicious proposal reverts atomically rather than partially filling. It enforces, and reverts on violation of, all of the following:

- nonce equals the expected value (replay protection) and current ledger time $\leq$ deadline;
- every adapter, asset_in and asset_out is on the admin-maintained allowlist;
- every leg carries min_amount_out at least as tight as the bound the contract recomputes from its own TWAP and the maximum-slippage cap, so a manipulated pool or a self-dealing leg fails rather than fills badly;
- aggregate notional is below a cap expressed as a fraction of NAV;
- a minimum cooldown has elapsed since the last rebalance, and the vault is not paused.

```
#[contracterror]
pub enum Error {
    Paused = 1, NotAuthorized = 2, BadNonce = 3, Expired = 4, Cooldown = 5,
    AssetNotAllowed = 6, AdapterNotAllowed = 7, Slippage = 8, NotionalCap = 9,
    TwapUnavailable = 10, OracleStale = 11, OracleDivergence = 12, ZeroShares = 13,
}

pub fn execute_rebalance(e: Env, plan: Plan) {
    require_role(&e, DataKey::Executor); // 1. authorized proposer only
    if paused(&e) { panic_with_error!(&e, Error::Paused); }

    let now = e.ledger().timestamp();
    if plan.nonce != next_nonce(&e) { panic_with_error!(&e, Error::BadNonce); } // replay
    if now > plan.deadline { panic_with_error!(&e, Error::Expired); } // freshness
    if now < last_rebalance(&e) + cooldown(&e) { panic_with_error!(&e, Error::Cooldown); }

    guard_pricing(&e); // 2. oracle sanity (next section)

    let nav0 = nav(&e);
    let mut notional: i128 = 0;
    for leg in plan.legs.iter() { // 3. per-leg invariants
        if !asset_ok(&e, &leg.asset_in) { panic_with_error!(&e, Error::AssetNotAllowed); }
        if !asset_ok(&e, &leg.asset_out) { panic_with_error!(&e, Error::AssetNotAllowed); }
        if !adapter_ok(&e, &leg.adapter) { panic_with_error!(&e, Error::AdapterNotAllowed); }
        // contract recomputes the floor from TWAP; the proposer cannot widen it
        let floor = twap_min_out(&e, &leg) * (BPS - max_slip_bps(&e)) / BPS;
        if leg.min_amount_out < floor { panic_with_error!(&e, Error::Slippage); }
        notional += value_in(&e, &leg);
    }
    if notional > nav0 * max_turnover_bps(&e) / BPS { // 4. aggregate notional cap
        panic_with_error!(&e, Error::NotionalCap);
    }

    for leg in plan.legs.iter() { // 5. execute via allowlisted adapter
        adapter::swap(&e, &leg); // honors the on-chain min_amount_out
    }
    bump_nonce(&e);
    set_last_rebalance(&e, now); // 6. commit + emit
    e.events().publish((symbol_short!("rebal"), executor(&e)),
        (plan.nonce, plan.hash(&e), plan.legs.len() as u32));
}
```

Because min_amount_out is recomputed on-chain from the TWAP and a fixed maximum-slippage parameter, a backend that proposes an off-market or self-dealing leg simply fails the slippage check: it cannot widen its own bound, and it never holds withdrawal rights to begin with.

8.6 Pricing layer and oracle trust model

NAV and execution bounds use a two-layer design, and the contract owns both reads. The executor never supplies a price the contract trusts.

- **Primary: Soroswap TWAP.** Time-weighted average prices from allowlisted Soroswap pools value NAV and derive every `min_amount_out`. Using TWAP rather than spot means a single-block pool push cannot move the bound.
- **Circuit breaker: Reflector.** The Reflector feed is compared against the TWAP; divergence beyond a threshold, or staleness on either source, halts risk-increasing execution.

The failure modes are handled explicitly, and the design deliberately separates *valuation liveness* (needed to price deposits) from *execution liveness* (needed to rebalance) from *withdrawal*, which depends on neither:

```
fn guard_pricing(e: &Env) {
  let now = e.ledger().timestamp();
  for asset in assets(e).iter() {
    match pricing::soroswap_twap(e, &asset) { // Option<Quote>
      None => panic_with_error!(e, Error::TwapUnavailable), // halt rebalancing
      Some(q) if now - q.ts > MAX_AGE =>
        panic_with_error!(e, Error::OracleStale),
      Some(q) => match pricing::reflector(e, &asset) {
        Some(r) if now - r.ts <= MAX_AGE => {
          let dev = (q.price - r.price).abs() * BPS / r.price;
          if dev > MAX_DEVIATION_BPS { // sources disagree
            panic_with_error!(e, Error::OracleDivergence);
          }
        }
        _ => panic_with_error!(e, Error::OracleStale), // breaker unavailable
      }
    }
  }
}
```

Pricing condition	Rebalance	De-risk leg	Deposit / Redeem
TWAP live, Reflector within band	allowed	allowed	allowed
TWAP stale or unavailable	halt	guardian-only	deposit halts; redeem OK
TWAP vs Reflector divergence > threshold	halt	guardian-only	deposit halts; redeem OK
Both sources stale / unavailable	halt	halt	redeem OK (in-kind)

The invariant across the table: pro-rata redeem is in-kind and never gated on price liveness or backend health, so the worst an oracle problem can do is freeze *rebalancing* and new deposits, which are exactly the actions that could lose money if priced wrong. This matches the executor trust model: the backend is a proposer, the contract is the program, and every dangerous path is refused on-chain.

9 Threat Model and Safeguards

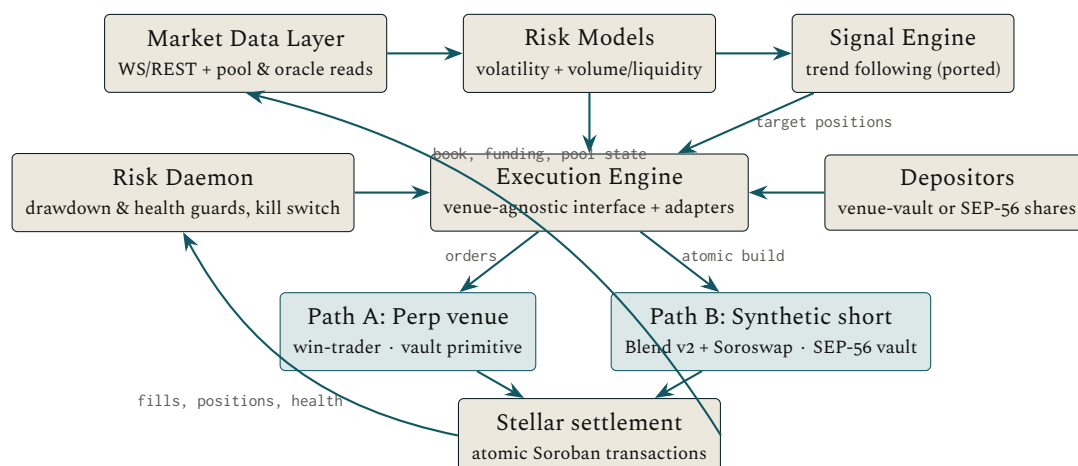
Scenario	Mitigation
Backend compromise	Executor cannot withdraw; plans outside allowlists, slippage, notional or cooldown bounds revert on-chain. Blast radius is a bad-but-bounded rebalance.
Pool price manipulation	<code>min_amount_out</code> computed from Soroswap TWAP, not spot; Reflector deviation or staleness halts execution.
Backend outage	Withdrawals are permissionless and unaffected. Vault simply stops rebalancing; guardian can de-risk. Redundant keepers across independent infrastructure reduce liveness risk.
Oracle staleness	Reflector staleness checks halt execution rather than price NAV off stale data.
Reentrancy	State is committed (nonce bump, share supply, last-rebalance) before external token and adapter calls; Soroban's per-call authorization and the absence of implicit fallback hooks remove the classic reentrancy surface.
Key compromise (admin)	Parameter changes are timelocked; guardian pause limits the window.
Contract bugs	Rust type safety, Soroban auth patterns, OpenZeppelin Stellar Suite components, invariant-focused unit tests ($\geq 90\%$ coverage target), third-party audit before mainnet (audit credits provided through the SCF program).

10 Phase 2: Long-Short Systematic Vault

The long-short vault (Vault 2) is the full long and short CTA, porting Etesia’s live trend-following strategy from Hyperliquid to Stellar and adding the short side that spot DEX execution cannot provide. It is built behind a *venue-agnostic execution interface* with two interchangeable paths:

- **Native perpetuals.** Directional exposure via whichever Stellar perp venue reaches mainnet first. win-trader (formerly stellars.finance; SCF grant delivered, in final audit, mainnet planned before September 2026) is the primary candidate, with Zenex and Noether monitored as alternatives. Custody, deposits and settlement use the venue’s vault primitive on Stellar.
- **Synthetic shorts via Blend v2 + Soroswap.** Short exposure built atomically on Stellar: deposit collateral on Blend v2, borrow the target asset, swap it on Soroswap, with effective leverage from recursive collateralization (detailed below). Carry cost (the Blend borrow rate) is comparable to perp funding. Custody sits in an Etesia SEP-56 Soroban vault, the same primitive as the long-only vaults.

The signal engine, risk controls and position sizing are *identical* across both paths; the venue is a thin adapter, not a second build. If no native perp venue reaches mainnet on our timeline, the synthetic-short path delivers the same exposure, so the Phase 2 deliverables do not hinge on any single third-party venue going live. Signal generation and execution run from our off-chain Python infrastructure, exactly as in our Hyperliquid deployment. The hard engineering problem is not the strategy, which is proven and running, but the execution infrastructure (clean market data, liquidity statistics, execution tooling) built and calibrated to a thin, growing on-chain market.



10.1 Market data layer

Data ingestion is path-dependent but normalizes into a single internal schema (shared with our Hyperliquid stack), so every model downstream operates identically regardless of execution path.

- **Native perp path.** Custom connectors against the venue’s WebSocket and REST APIs ingest the order book (depth snapshots and incremental updates), trades, funding rates, and mark/index prices.
- **Synthetic-short path.** Reads from Soroswap pools (prices, depth, volume) and Blend v2 pool state (utilization, borrow rates, available liquidity, collateral factors), normalized into the same schema.

Data is persisted (time-series storage with Parquet archival) with sequence-gap detection, automated backfill and cross-source consistency checks. This layer also produces the recorded datasets against which the models below are calibrated and validated.

10.2 Volatility model

Position sizing requires a per-asset volatility estimate that is responsive but stable on a venue with limited history. Variance is estimated with a standard exponentially weighted estimator:

$$\hat{\sigma}_t^2 = \lambda \hat{\sigma}_{t-1}^2 + (1 - \lambda) r_t^2, \quad \lambda \in (0, 1).$$

We refine this with our own multi-horizon blending (proprietary, not detailed here) and, while native venue history is short, anchor estimates on the same asset’s price history from established venues, transitioning

weight to venue-native data as it accumulates. Volatility estimates feed both signal scaling and per-asset volatility-targeted position sizing.

10.3 Volume and liquidity model

On a young exchange, or a synthetic construction path, the binding constraint is what can be traded without moving the market. From recorded data we estimate rolling average daily volume, intraday volume curves, top-of-book and cumulative depth profiles, and an empirical impact curve of square-root form

$$\text{cost}(Q) \approx \frac{1}{2} \text{spread} + \eta \hat{\sigma} \sqrt{Q/V},$$

with Q the order size and V the relevant volume. On the native perp path, V is the venue's traded volume; on the synthetic-short path, V is derived from Blend v2 available liquidity and Soroswap pool depth. The model outputs per-asset participation caps and maximum position sizes, which bound the signal engine's targets so the strategy never demands liquidity the execution path cannot provide.

10.4 Execution engine

The execution engine translates target positions into actions through a venue-agnostic interface (the same abstraction our Hyperliquid engine uses), with two thin adapters behind it:

- **Perp venue adapter.** Order placement and amendment, slicing within participation limits, passive-first placement with re-quoting and escalation to marketable orders on urgency, fill tracking, and continuous reconciliation of internal state against venue-reported positions and balances.
- **Synthetic-short adapter.** Atomic Soroban transactions combining a Blend v2 collateral deposit and borrow with a Soroswap swap (a single multi-invoke), managing recursive collateralization to reach the target leverage, and unwinding via the reverse path with the same atomicity.

Both adapters implement the same interface, so the strategy code above is unchanged when switching paths.

10.5 Signal engine and risk controls

The signal engine is the trend-following engine described above, here producing long/short target positions per asset scaled to the portfolio volatility target. An independent risk daemon enforces drawdown guards, leverage and position limits, data-staleness checks, and a kill switch that flattens positions and halts trading; it runs separately from the execution path so a strategy fault cannot disable risk enforcement.

On the synthetic-short path the risk daemon additionally monitors Blend v2 health factors and triggers deleveraging before the liquidation threshold is reached. This replaces the funding-rate monitoring of the perp path: same economic function (cost-of-carry and solvency management), different metric.

10.6 Vault integration

Custody is path-dependent, and both options keep depositors non-custodial:

- **Native perp path.** Depositors access the strategy through the venue's vault primitive: deposits, share accounting, and profit attribution are handled on Stellar, while our infrastructure trades the vault's account. Funds remain withdrawable through the venue independently of our infrastructure. This mirrors the operating model of our live Hyperliquid vault.
- **Synthetic-short path.** Depositors use an Etesia SEP-56 Soroban vault, the same primitive as the long-only vaults (see On-Chain Design). The orchestrator executes construction and unwinding on the vault's behalf under the same on-chain guardrails; funds are withdrawable permissionlessly through the vault contract with no dependence on our backend being online.

10.7 Synthetic short construction

The synthetic short is a standard DeFi primitive, executed atomically on Stellar:

1. Deposit collateral (Asset A, e.g. USDC) into a Blend v2 pool.
2. Borrow the target Asset B against that collateral, subject to the pool's LTV.

3. Swap $B \rightarrow A$ on Soroswap.

4. Re-deposit A for recursive collateralization, reaching effective $\sim 2-3\times$ exposure depending on LTV.

Steps 1–3 execute as a single atomic multi-invoke Soroban transaction; unwinding reverses the path with the same atomicity. The **carry cost** is the Blend borrow rate on B minus the lending yield on A, comparable to perp funding. The **health factor** is monitored continuously, and approaching the liquidation threshold triggers an automatic partial unwind (via the risk daemon above). The tradable universe is limited to assets with sufficient Blend liquidity and Soroswap depth, enforced through the per-asset participation caps of the volume and liquidity model above.

11 Technology Stack

Layer	Technology
Smart contracts	Rust + Soroban SDK; SEP-56 / SEP-41; OpenZeppelin Stellar Suite; Scaffold Stellar.
Quant / strategy	Python (NumPy, pandas, cvxpy-class solvers), shared codebase with the live Hyperliquid deployment.
DEX routing	Soroswap routing API (primary), Phoenix (secondary); also the swap leg of the synthetic-short path.
Yield / synthetic short	Blend v2 lending pools (USDC, EURC) for idle-stablecoin yield; collateral-deposit and borrow legs of the long-short vault's synthetic-short path.
Composability	DeFindex Strategy interface (invest, unwind, harvest).
Frontend	Next.js + React, Tailwind CSS; Stellar Wallets Kit (Freighter, xBull).
Pricing	Soroswap TWAP (primary) + Reflector oracle (circuit breaker).
Orchestration	Node.js / TypeScript; job scheduling; PostgreSQL event indexing; Parquet archival.
Testing	Rust native contract tests, invariant tests; Python backtest and replay harness; testnet stress scenarios.

12 Mapping to Build Award Tranches

Each tranche names at least one integration from the SCF Integration List. On-list integrations are marked in bold.

Tranche	Scope	On-list
Tranche 1 (MVP)	Trend-following signal engine and risk parity allocation engine with back-testing; SEP-56 vault contracts with on-chain guardrails; Soroswap TWAP pricing as the on-chain execution reference, with Reflector as the deviation and staleness circuit breaker.	Soroswap (TWAP)
Tranche 2 (Testnet)	Orchestrator with scheduled and signal/drift-triggered rebalancing; Soroswap swap execution on testnet with Phoenix as a secondary path; idle stablecoins deposited into Blend v2 lending pools with yield accruing to NAV; Portfolio Builder tool and vault data indexer with Stellar Wallets Kit connectivity; stress testing, monitoring and mainnet readiness.	Soroswap (swaps), Blend v2 , Wallets Kit
Tranche 3 (Mainnet)	Mainnet launch of the long-only vault and Portfolio Builder with Soroswap rebalancing and Blend yield live; vaults expose the DeFindex Strategy interface for composability; then the long-short execution infrastructure (market data, volatility and volume models) and launch of the Long-Short Systematic Vault via a native perp venue (win-trader primary) or synthetic shorts on Blend v2 + Soroswap .	DeFindex , Blend v2 , Soroswap